

ABSTRACT DATA TYPES ***(ADT)***

Linear Lists

- ❑ Since we are using C, we adopt the convention that positions start at zero. This will mean that if we have n elements, the last element is at position $n-1$. In some other languages e.g. Pascal, positions start at 1.
- ❑ A list is a sequence of zero or more elements of a given type (which we generally call the *elementtype*)
- ❑ Such a list is usually represented as a comma separated sequence of elements i.e. $a_0, a_1, a_2, \dots, a_{n-1}$ where $n \geq 0$.
- ❑ The number of elements n is said to be the length of the list.
- ❑ Assuming $n \geq 1$, we say that a_0 is the first element and a_{n-1} is the last element.

Linear Lists

- If $n=0$, then we have an empty list i.e one with no elements
- An important property of a list is that its elements can be linearly ordered according to their position in the list
- We say that a_i precedes a_{i+1} for $i=0,1,\dots,n-2$
- We say that a_i follows a_{i-1} for $i=1,2,3,\dots,n-1$
- Element a_i is said to be in position i .
- The position following the last position of the list can be termed as $end(l)$, where l is the list and $end(l)$ is a function that returns the position after the last. In C, it can be seen that $end(l)=n$.

Operations on linear lists

- ☐ Access the element in position k .
- ☐ Insert new element into list at position k
- ☐ Delete a specified k -th element (Deletion)
- ☐ Combine two or more lists into a single list (Merging)
- ☐ Divide the list into two or more lists
- ☐ Sort the List according to the values of field in the elements (Sorting)
- ☐ Search for an element in which a certain field has a given value (Searching)

Examples of List Operations.

L1 = (dog,cat,rat,bat)

InsertA(mouse,Locate(rat,L1),L1)

L1 = (dog,cat,rat,mouse,bat)

Delete(First(L1),L1)

L1 = (cat,rat,mouse,bat)

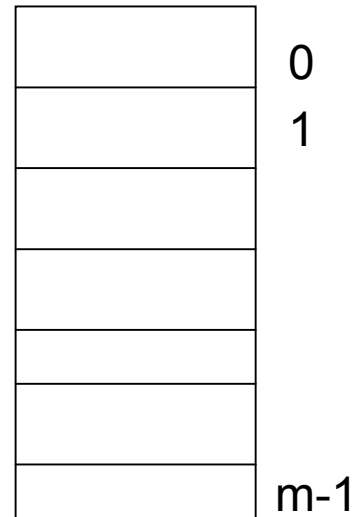
L2 = (apple,orange,pear,banana)

Concatenate(L1,L2)

L1 = (cat,rat,mouse,bat,apple,orange,pear,banana)

Lists using Arrays

- ❑ Suppose we want to store the elements of a list sequentially in a computer.
- ❑ One method to do this is using a one dimensional array
- ❑ To get the storage locations a **one-dimensional array** is set-up forming a series of locations:
- ❑ The subscript of the of the array i.e. $V[0]$, $V[1]$ are the address of the locations. If the elements are $(a_0, a_1 \dots a_{n-1})$, then it is an n -length list elements of the list can be stored sequentially.
- ❑ $0, 1, 2, 3 \dots n-1$ are the addresses of the location for each element



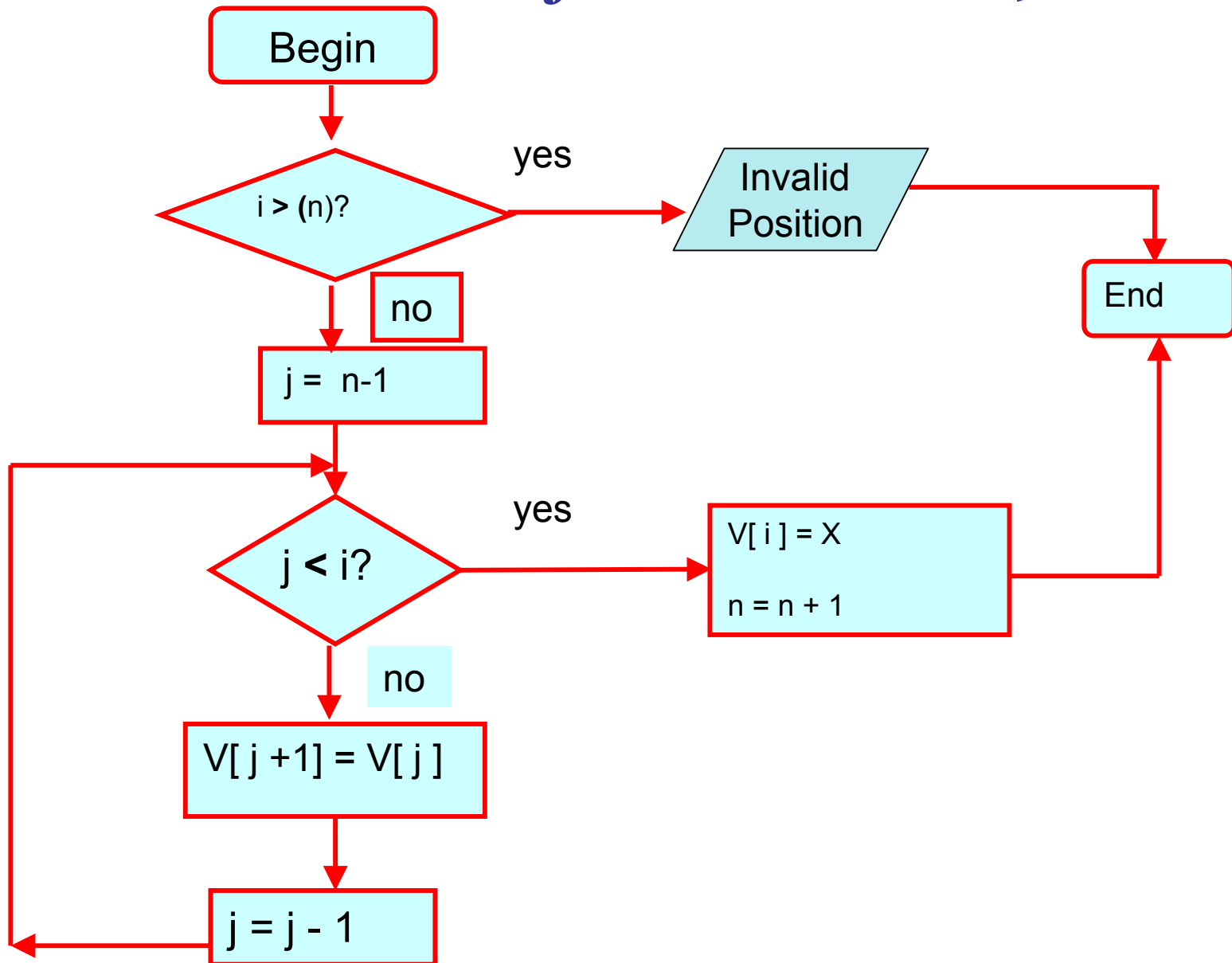
*Sequential
Allocation*

Lists using Arrays

List insertion

- The insertion operation requires that we insert a new element x at position i of the list.
- To achieve this a_i through a_{n-1} are shifted one position up the list so that element a_i moves to position of $a_i + 1$ and a_{n-1} to position of a_n
- A total of $(n - i)$ elements are shifted.
- An algorithm for the procedure can be represented in a flow chart (next slide)

(Insert Element x into position i in the list (V), n is the number of items in the list)



Insertion into a List-algorithm

- ❑ First step if $i > n$ then error is flagged; since we only have n elements, a new element can only be inserted at position n or less.
- ❑ Next an initial value is assigned to j i.e. $j = n-1$.
- ❑ Then compare j and i . If $j \geq i$ then move a_j (i.e. $V[j]$) to position $j+1$ (i.e. $V[j+1]$).

Insertion into a List-algorithm

- ❑ The first to be moved is $V[n-1]$. Then change index i.e. j changes to $j-1$, compare j and i and the shifting continues until $j = i$
- ❑ Then $V[i] = V[i+1]$, that is i moved to $(i+1)$ th position.
- ❑ Next comparison will stop since $j = i-1$ hence less than i
- ❑ At this point i th position is vacant for which next operation is performed $V[i] = x$ that is x is inserted in position i and length of list is increased to $n+1$ from n .
- ❑ End of Insertion steps

Insertion into a List-C Program

```
#include <stdio.h>
#define MAX 10
int V[MAX] = {1,23,17,4,-5,100};
int n=6, j;
// Inserting function
void list_insert(int
    listname[],int x,int i)
{
    if(i>n || n>MAX-1)
    {
        printf("\nOverflow error \n");
        return;
    }
    else if(i<0)
    {
        printf("\ninvalid position\n");
        return;
    }
    j=n-1;
    while(j>=i)
    { listname[j+1]=listname[j];
      j=j-1;
    }
    listname[i] = x;
    n=n + 1;
}
```

```
int main(void)
{
    int i=0,x=0,j=0;
    printf("Enter your position <%d\n",MAX-1);
    scanf("%d",&i);
    printf("Enter your element \n");
    scanf("%d",&x);
    printf("Position is %d\n",i);
    printf("Element is %d\n",x);
    printf("\n\n");
    printf("***** Before Insertion
***** \n");
    printf("\n\n");
    for (j = 0; j < n; j++)
    {
        printf("V[%d] = %d    ",j,V[j]);
    }
    // Call insert element function
    list_insert(V,x,i);
    printf("\n\n");
    printf("***** After Insertion
***** \n");
    printf("\n\n");
    for (j = 0; j < n; j++)
    {
        printf("V[%d] = %d    ",j,V[j]);
    }
    return 0;
}
```

Lists- Insertion: C program Expected output

C:\CONTENTS FROM D\COURSES\ICS 605\CHEPKEN\EXTRA NOTES\Lists\Debug\Lists.exe"

Enter your position

3

Enter your element

27

Position is 3

Element is 27

***** Before Insertion *****

U[0] = 1 U[1] = 23 U[2] = 17 U[3] = 4 U[4] = -5 U[5] = 100

***** After Insertion *****

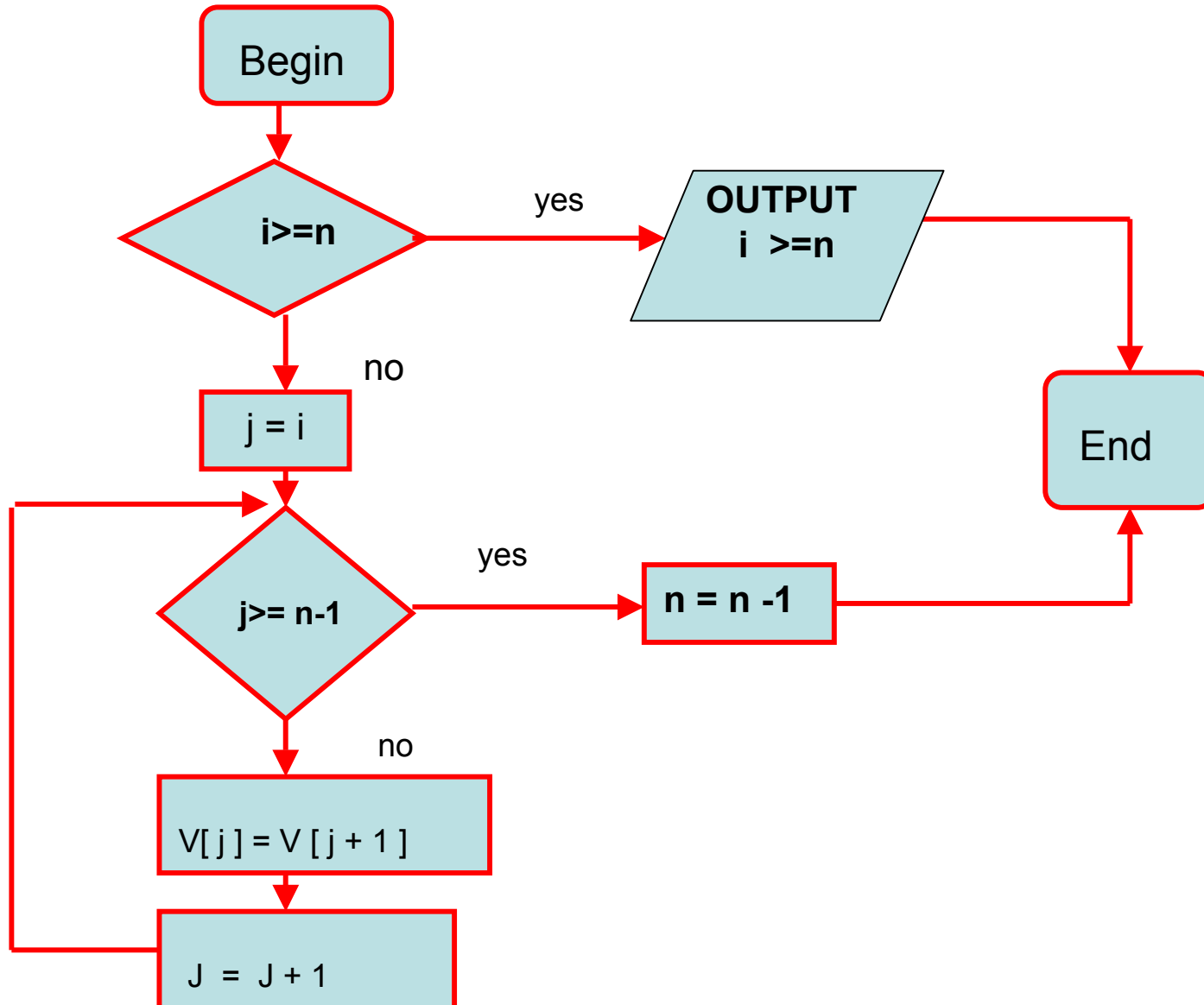
U[0] = 1 U[1] = 23 U[2] = 17 U[3] = 27 U[4] = 4 U[5] = -5 U[6] = 100

Press any key to continue_

Delete an element from a list

- ❑ Involves deleting an element a_i (at position i) from a list $(a_0, a_1, \dots, a_{n-1})$.
- ❑ After deleting the i^{th} element, elements in $(i+1)$ position to position $n-1$ are shifted one position backwards and length of list adjusted to $n-1$.
- ❑ Total of $(n-i-1)$ elements are moved backwards.

Delete an element from a list



Delete an element from a list

❑ *Algorithm Steps.*

- i and n are compared
- if $i \geq n$ then error is flagged (the last element is at position $n-1$)
- Else assign i to j (initial value of index j).
- Next compare j and $n-1$ till $j \geq n-1$, end if j is greater or equal to $n-1$ (we have shifted all elements)
- Else shift $V[j+1]$ to $V[j]$ then increment j to $j+1$ and loop

Lists- Deletion:C program

```
#include <stdio.h>
#define MAX 10
int V[MAX] = {1,23,17,4,-5,100};
int n=6, j;

// Deleting function
void list_delete(int listname[],int i)
{
    int j;
    if (i>=n)
    {
        printf("\nError in deleting \n");
        return;
    }
    j=i;
    while(j<=n-1)
    {
        listname[j]=listname[j+1];
        j=j+1;
    }

    n=n-1;
}
```

```
int main(void)
{
    int i=0,j;
    printf("Enter your position \n");
    scanf("%d",&i);
    printf("Position is %d\n",i);
    printf("\n\n");
    printf("***** Before deleting ***** \n");
    printf("\n\n");
    for (j = 0; j < n; j++)
    {
        printf("V[%d] = %d  ",j,V[j]);
    }
    // Call delete element function
    list_delete(V,i);
    printf("\n\n");
    printf("***** After deleting ***** \n");
    printf("\n\n");

    for (j = 0; j < n; j++)
    {
        printf("V[%d] = %d  ",j,V[j]);
    }
    return 0;
}
```

Lists- Deletion:C program expected output

 "C:\CONTENTS FROM D\COURSES\ICS 605\CHEPKEN\EXTRA NOTES\Lists\Debug\Lists.exe"

Enter your position

2

Position is 2

***** Before deleting *****

U[0] = 1 U[1] = 23 U[2] = 17 U[3] = 4 U[4] = -5 U[5] = 100

***** After deleting *****

U[0] = 1 U[1] = 23 U[2] = 4 U[3] = -5 U[4] = 100

Press any key to continue

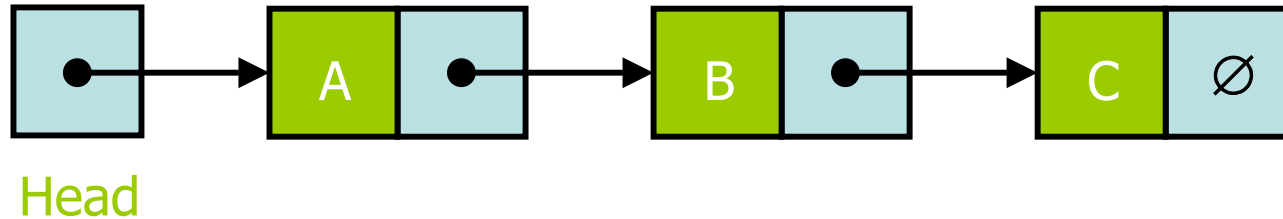
Linked lists

- ❑ The linked list is a very flexible **dynamic data structure**: items may be added to it or deleted from it at will.
- ❑ An array (linear list) allocates memory for all its elements lumped together as one block of memory.
- ❑ In contrast, a linked list allocates space for each element separately in its own block of memory called a "linked list element" or "node".

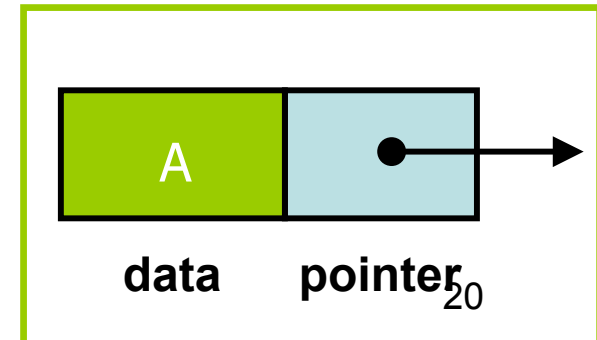
Linked lists

- ❑ The list gets its overall structure by using **pointers** to connect all its nodes together like the links in a chain.
- ❑ Each node contains two fields: a "data" field to store whatever element type the list holds for its client, and a "next" field, which is a pointer used to link one node to the next node.

Linked lists



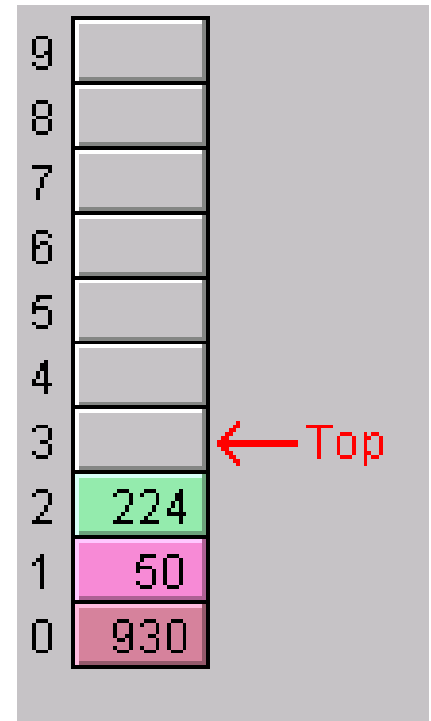
- ❑ A *linked list* is a series of connected *nodes*
- ❑ Each node contains at least
 - A piece of data (any type)
 - Pointer to the next node in the list
- ❑ *Head*: pointer to the first node_{node}
- ❑ The last node points to NULL



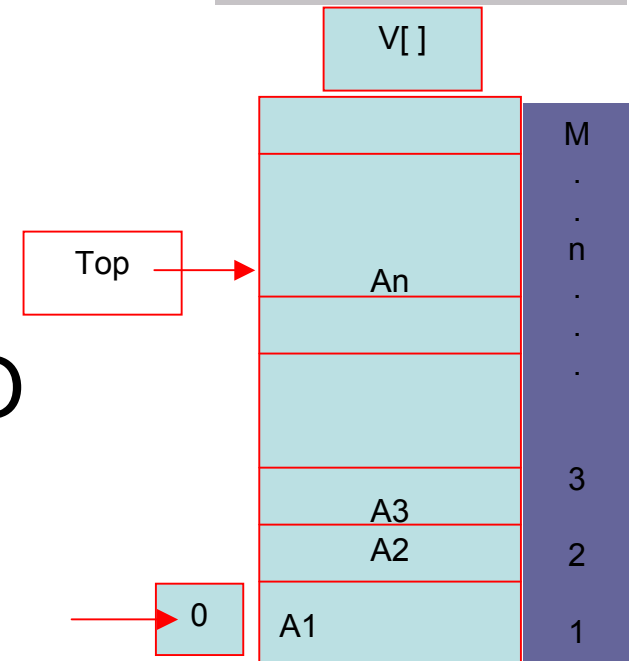
STACKS AND QUEUES

Stacks: Definition:

- ❑ Is a Linear list in which all insertions and deletions are made at one end called the Top. E.g. in Linear List $\langle a_1 a_2 \dots a_n \rangle$ deletion and insertion can only be on element a_n (Top element).



- ❑ It is a LIFO (Last In First Out) list also known as FILO (First In Last Out)



Stacks

- ❑ The storage is downwards-up so that all operations are on Top element.
- ❑ A pointer is restricted to the Top element.
- ❑ If there is no element in stack ($n=0$.) The top will be -1

Note:

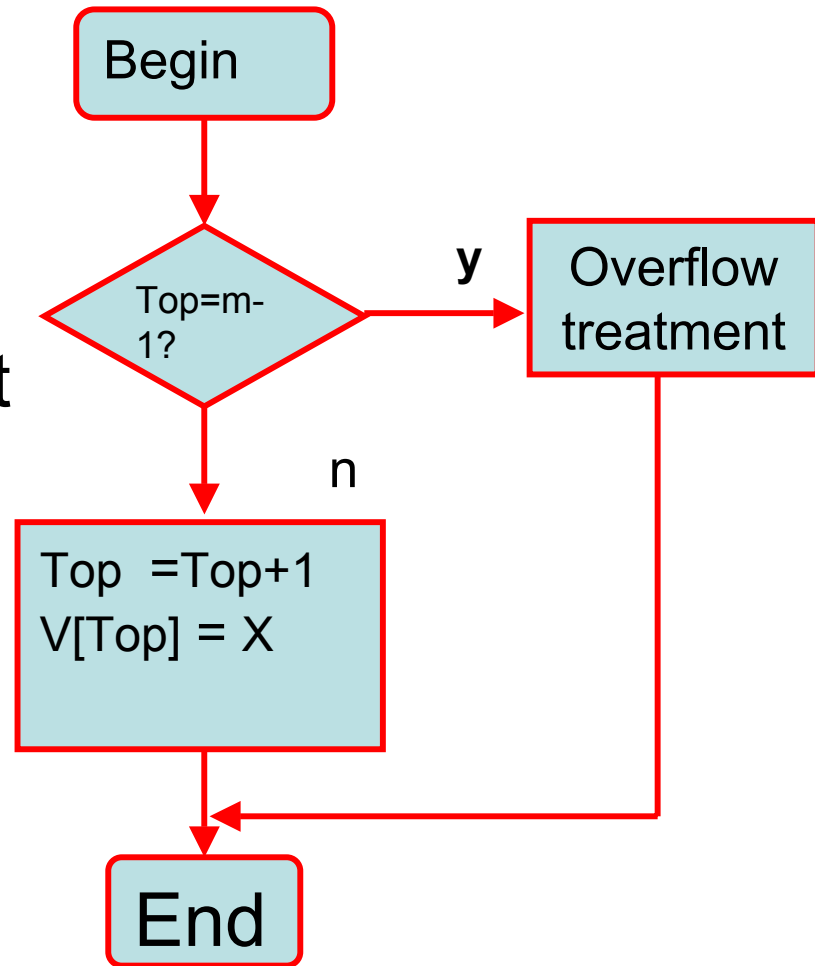
- ❑ Index used from 0 i.e. $V[0] \dots V[n-1]$.
- ❑ [Top = -1 means empty Stack]
- ❑ [Top = $M-1$ means full Stack] for an M -capacity storage Stack.

Stack operations

- ❑ `makeNull(s)` – make `s` be an empty stack
- ❑ `top(s)` – return the element at the top of the stack
- ❑ `pop(s)` – return and delete the element at the top of the stack. The stack size reduces by 1
- ❑ `push(x,s)` – insert element `x` at the top of the stack. The size of the stack increases by one.
- ❑ `empty(s)` – returns true if the stack has no elements.

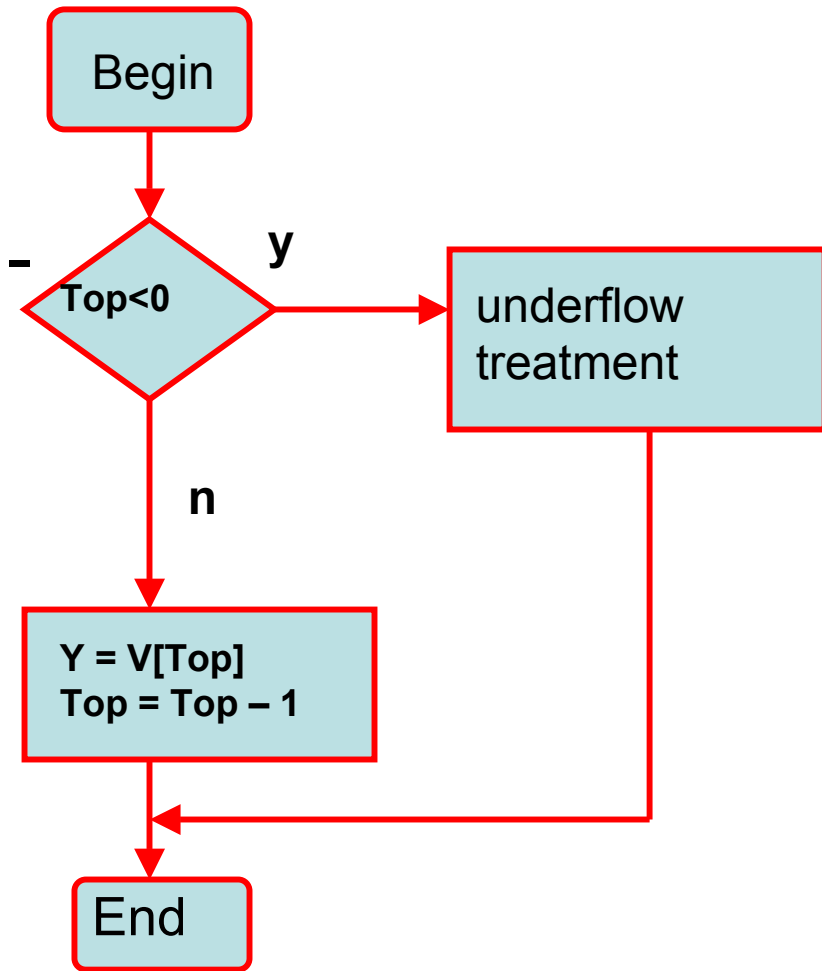
Insertion in a stack

- ❑ For example inserting new element say X (Fig)
- ❑ A test is made of whether Stack is full: If full then abort the procedure or else move pointer Top to position Top+1 then insert element X, i.e. $V[\text{Top}] = X$
- ❑ X becomes the new top element:



Stack:Deleting an element (Top element)

- ❑ Declare temporary variable Y is to store the deleted element:
- ❑ If Stack is empty i.e. $\text{Top} = -1$ then there occurs what is known as an underflow.
- ❑ Otherwise delete the top element that is, $V[\text{top}]$, and adjust pointer Top to $(\text{Top}-1)$.
- ❑ The deleted element Y may be printed out



Stack Applications

- ❑ Reversing Data: We can use stacks to reverse data.
(example: files, strings)
- ❑ Very useful for finding palindromes (reading the same backward or forward).
- ❑ Consider the following pseudocode:
 - 1) read (data)
 - 2) loop (data not EOF and stack not full)
 - 1) push (data)
 - 2) read (data)
 - 3) Loop (while stack notEmpty)
 - 1) pop (data)
 - 2) print (data)

Stack Applications

❑ Converting Decimal to Binary: Consider the following pseudocode

- 1) Read (number)
- 2) Loop (number > 0)
 - 1) digit = number modulo 2
 - 2) print (digit)
 - 3) number = number / 2

// from Data Structures by Gilbert and Forouzan

The problem with this code is that it will print the binary number backwards. (ex: 19 becomes 11001000 instead of 00010011.)

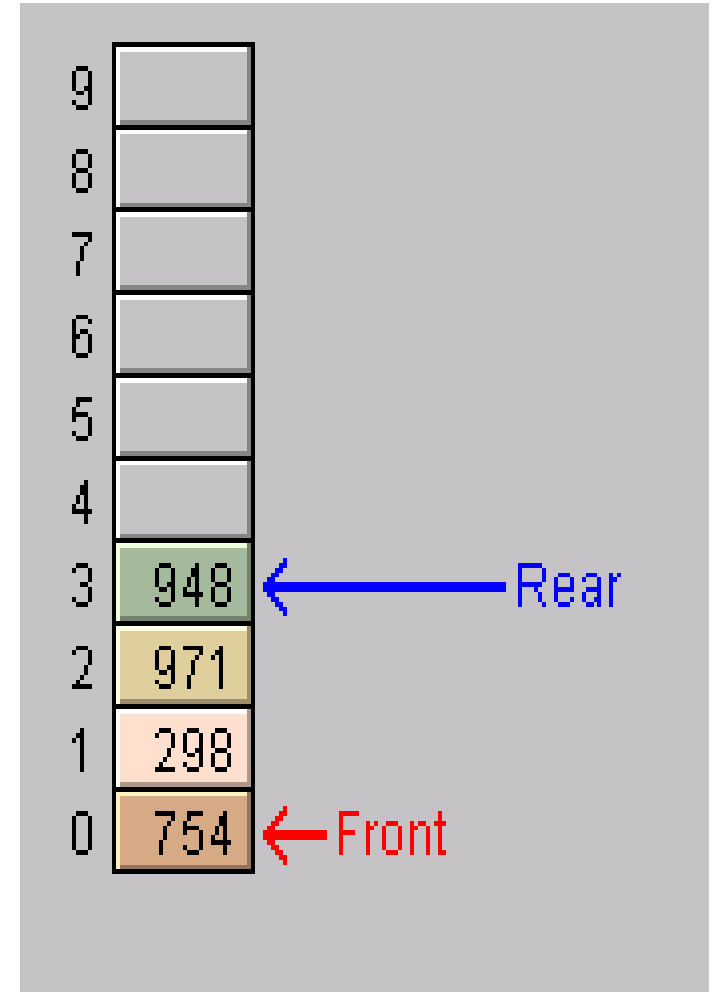
To remedy this problem, instead of printing the digit right away, we can

push it onto the stack. Then after the number is done being converted, we

pop the digit out of the stack and print it.

Queues: Definition:

- ❑ Is a linear list in which all insertions and deletions are restricted:
- ❑ All insertions into the queue take place at one end called the rear while all deletions take place at the other end called the front



Queue: Operations:

- ❑ **makeNull(q)** – makes a queue empty and returns an empty queue
- ❑ **front(q)** – returns the first element on a queue
- ❑ **enqueue(x,q)** – inserts element q at the end of the queue
- ❑ **dequeue(q)**– deletes the first element element of the queue
- ❑ **Empty(q)** – returns true iff the queue is empty.

Queues Insertion (enqueue); Example

- Insertion of element x
- For this, it's imperative to check whether queue is full or not.

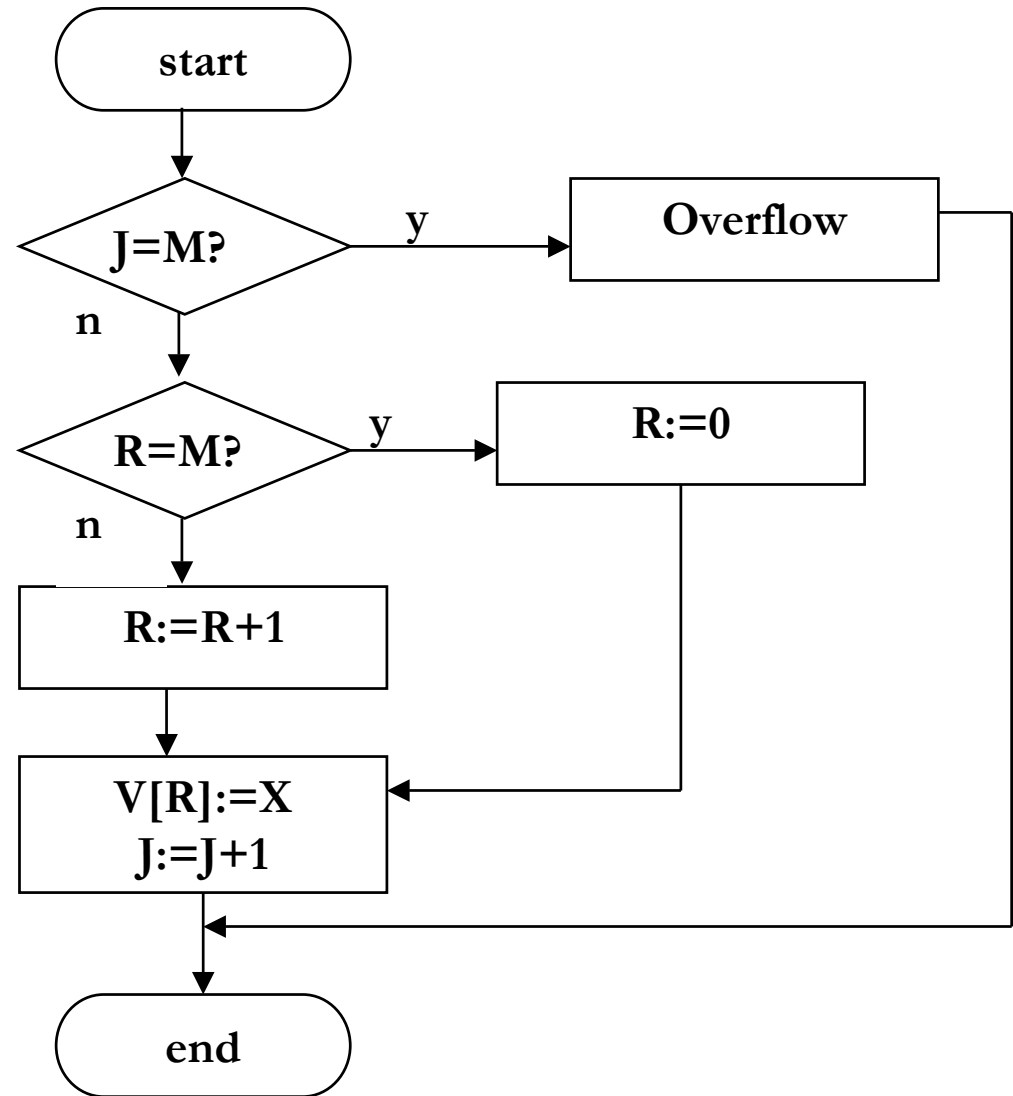
R = Rear points to rear Element

F = Front points to front element

M – size of the array

J – the number of elements currently in the list. its 0 if queue is empty and M if queue is full

Initially, R is -1 and F is 0.



Queues deletion/dequeue;Example

- Deletion of an element

- For this, it's imperative to check whether queue is empty or not.

R= Rear points to rear Element

F= Front will point to position after first element (after the deletion).

